

מחברת 5: בינה מלאכותית למדעי הרוח 🤖

מבוא למדעי הרוח הדיגיטליים | אוניברסיטת אריאל | תשפ"ו

פרופ' שי גורדין

מה נלמד במחברת זו? AI (Artificial Intelligence) = בינה מלאכותית – היכולת של מחשבים לבצע משימות הדורשות "חשיבה".

נושאים:

1. מהי AI ומה רלוונטי למדעי הרוח?
2. LLMs – מודלי שפה גדולים (ChatGPT, Claude)
3. Prompt Engineering – כיצד לשאול AI בצורה יעילה
4. שימוש ב-API של Claude לניתוח טקסטים
5. סיווג טקסטים עם AI
6. שיקולים אתיים

מדריך שימוש במחברת 🌐

||| |---|---| זמן מוערך | 🏠 בכיתה: ~60 דקות | 🏠 בבית: ~40 דקות | | 🖥️ היכן להריץ |
Google Colab | 🔑 מפתח API | אופציונלי – כל הקוד עובד ללא API (יש מצב סימולציה) |

שימוש חינמי ב-AI אמיתי – Google Gemini

המחברת כוללת אפשרות להשתמש ב-Google Gemini חינמית (ללא תשלום):

1. עברו ל-Google AI Studio
2. לחצו "Create API key" → "Get API Key"
3. העתיקו את המפתח
4. הדביקו אותו בתא המפתח למטה

⚠️ אל תשתפו מפתח API עם אף אחד – שמרו אותו רק לעצמכם!
⚠️ אל תכתבו מפתח API ישירות בקוד – השתמשו בשיטה מאובטחת (ראו למטה)

חלק א: AI ומדעי הרוח – מפגש מרתק

הגל השלישי של LLMs AI

LLM (Large Language Model) = מודל שפה גדול שאומן על כמויות עצומות של טקסט.

מודל	יוצר	שנה	מאפיין
GPT-4	OpenAI	2023	שוק מסחרי
Claude	Anthropic	2023	בטיחות-ממוקד
Gemini	Google	2024	מולטימודלי
Llama	Meta	2023	קוד פתוח

AI למדעי הרוח – אפשרויות:

- **תרגום** טקסטים עתיקים (ארמית, יוונית, לטינית)
- **סיכום** מסמכים ארוכים
- **סיווג** טקסטים לקטגוריות
- **זיהוי** דמויות, מקומות, תקופות
- **שאלות ותשובות** על מסמכים היסטוריים
- **ניתוח** שינויי שפה לאורך זמן

מגבלות חשובות:

- **הזיות (Hallucinations):** AI ממציא עובדות שאינן נכונות
- **Cutoff תאריך:** לא יודע אירועים חדשים
- **הטיות:** משקף הטיות בנתוני האימון
- **עלות:** שימוש ב-API עולה כסף לקריאה: Bender et al. (2021). *On the Dangers of Stochastic Parrots*. FAccT

```
In [ ]: # התקנת ספריות
# google-generativeai = Gemini ← עיקרי לשיעור
# anthropic / openai = Claude / GPT ← (אופציונלי)
!pip install google-generativeai anthropic openai -q

print("✅ הותקנו AI ספריות")
print()
print("🆓 Gemini (חינמי): https://aistudio.google.com/apikey")
print("💰 Claude (בתשלום): https://console.anthropic.com/")
print("💰 GPT (בתשלום): https://platform.openai.com/")
```

```
In [ ]: # יבוא ספריות
import os
import re
import json
import time
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
from IPython.display import display, HTML
import requests
import google.generativeai as genai
```

```

matplotlib.rcParams['axes.unicode_minus'] = False

# =====
# הגדרת מפתח API
# =====
# כאן Gemini אפשרות אי' (מומלצת לשיעור) - הדביקו את מפתח
GEMINI_API_KEY = "" # ← את המפתח שהורדתם מ aistudio.google.com

# (מאובטח יותר לפרויקטים אמיתיים) Environment Variable - אפשרות ב' - מ
# GEMINI_API_KEY = os.environ.get("GEMINI_API_KEY", "")

if GEMINI_API_KEY:
    genai.configure(api_key=GEMINI_API_KEY)
    print("✅ הוגדר Gemini מפתח")
else:
    print("⚠️ הנייד ישתמש בתגובות מדומות - Gemini לא הוגדר מפתח")
    print("🔗 קבלו מפתח חינמי: https://aistudio.google.com/apikey")

print()
print("✅ ספריות יובאו")
print()
print("⚠️ עם אחרים API כלל ברזל: לעולם אל תשתפו מפתח")
print("⚠️ GitHub-לעולם אל תעלו מפתחות ל")

```

חלק ב: Prompt Engineering – אמנות השאלה

מהו Prompt Engineering?

Prompt = ההנחיה שאתם שולחים ל-AI.

Prompt Engineering = אמנות ניסוח הנחיות יעילות.

עקרונות Prompt Engineering טוב:

עיקרון	תיאור	דוגמה
ספציפיות	הגדירו את המשימה בדיוק	"סכמו ב-3 משפטים" לעומת "סכמו"
הקשר	תנו רקע רלוונטי	"אתם ארכיאולוג המנתח..."
דוגמאות	הראו דוגמה לפלט רצוי	Few-shot prompting
פורמט	צינו את הפורמט הרצוי	"הגיבו ב-JSON" / "ב-רשימה מנוקדת"
תפקיד	הגדירו תפקיד ל-AI	"אתה חוקר היסטוריה המתמחה ב..."

:Chain of Thought

בקשו מה-AI להסביר את ההיגיון שלו: "חשבו שלב אחר שלב לפני שתענו..."

```

In [ ]: # =====
# Prompt Engineering דוגמאות

```

```

# =====

# -- API פונקציה לדמיית קריאת --
def simulate_llm_response(prompt, context=""):
    """
    (אמיתי API ללא LLM) פונקציה שמדמה תגובות

    API: בפרויקט אמיתי: תחליפו בקריאת

    import anthropic
    client = anthropic.Anthropic(api_key=os.environ['ANTHROPIC_API_KEY'])
    message = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=1024,
        messages=[{"role": "user", "content": prompt}]
    )
    return message.content[0].text
    """

    # תגובות מדומות לדוגמה
    demo_responses = {
        'summarize': """
        **סיכום (3 משפטים)**
        תל מגידו הוא אתר ארכיאולוגי חשוב בצפון ישראל, שנחפר מהמאה ה-19.
        הממצאים מציגים 26 שכבות התיישבות מהניאוליתי ועד לתקופה הפרסית.
        האתר זוהה עם המגידו המקראית, אחד מהאתרים האסטרטגיים החשובים בארץ ישראל.
        """,
        'classify': """
        {
            "קטגוריה": "ארכיאולוגיה",
            "תת_קטגוריה": "ארכיאולוגיה ביבשה",
            "תקופה": "ברונזה וברזל",
            "ביטחון": "0.92",
            "הסבר": "הטקסט עוסק בחפירות ארכיאולוגיות ושכבות התיישבות"
        }
        """,
        'translate': """
        **תרגום לעברית מודרנית**
        "ובשנה השלישית למלכותו של יאשיהו מלך יהודה,
        הגיע נבוכדנאצר מלך בבל לירושלים ויצור עליה."
        """
    }

    if 'סיכום' in prompt or 'סיכום' in prompt:
        return demo_responses['summarize']
    elif 'סיווג' in prompt or 'classify' in prompt.lower():
        return demo_responses['classify']
    elif 'תרגום' in prompt or 'תרגום' in prompt:
        return demo_responses['translate']
    else:
        return f"תגובה מדומה לפרומפט: {prompt[:50]}..."

# -- Prompts דוגמאות --
print("📦 Prompt Engineering:")
print("=" * 60)

```

```

prompts = {
    "❌ Prompt: גרוע": "ספר לי על ארכיאולוגיה",

    "✅ Prompt: טוב": """אתה ארכיאולוג המתמחה בארץ ישראל.
    סכם את הטקסט הבא ב-3 משפטים בעברית פשוטה:

    '...תל מגידו הוא אתר ארכיאולוגי'

    """,

    "✅✅ Few-shot": """דוגמה 1: "פירות בתל מגידו" <- "קטגוריה": "ארכיאולוגיה"
    דוגמה 2: "דוד המלך ירושלים" <- "קטגוריה": "היסטוריה"

    "" -> "סוג: כלים דיגיטליים לניתוח טקסט"
}

for title, prompt in prompts.items():
    print(f"\n{title}:")
    print(f" Prompt: {prompt[:80]}...")
    response = simulate_llm_response(prompt)
    print(f" תגובה: {response[:100].strip()}...")

```

```

In [ ]: # =====
# 🆓 Gemini-קריאה ל (חינמי - גוגל!)
# =====

def call_gemini(prompt, model_name="gemini-1.5-flash"):
    """
    ומחזיר את תגובת המודל Gemini-שולח פרומפט ל.

    (simulate_llm_response) מחזיר תגובה מדומה - GEMINI_API_KEY אם לא הוגדר

    פרמטרים:
        prompt - AI-הטקסט שרוצים לשלוח ל
        model_name - שם המודל (ברירת מחדל: gemini-1.5-flash, חינמי)

    דוגמה:
        תגובה = call_gemini("תל מגידו הוא...")
        print(תגובה)
    """

    if not GEMINI_API_KEY:
        print("💡 משתמש בתגובה מדומה - API אין מפתח")
        return simulate_llm_response(prompt)

    try:
        model = genai.GenerativeModel(model_name)
        response = model.generate_content(prompt)
        return response.text
    except Exception as e:
        print(f"❌ שגיאה: {e}")
        print("משתמש בתגובה מדומה במקום")
        return simulate_llm_response(prompt)

```

```
# -- בדיקת חיבור --
print("🐛 בודק חיבור ל-Gemini...")
test_response = call_gemini("מה 2+2 בעברית?")
print(f"Gemini: {test_response.strip()[:120]}")
print()
print("✅ מוכנה לשימוש call_gemini() הפונקציה")
```

חלק ג: שימוש ב-Claude API

מה זה Claude?

Claude הוא מודל שפה של Anthropic – פותח עם דגש על בטיחות ודיוק. מתאים במיוחד לניתוח טקסטים ארוכים, תרגום, וסיכום מסמכים היסטוריים.

מבנה ה-API:

```
import anthropic

client = anthropic.Anthropic(api_key="YOUR_KEY")

message = client.messages.create(
    model="claude-opus-4-5",      # המודל
    max_tokens=1024,             # מקסימום טוקנים בתגובה
    system="אתה חוקר היסטוריה...", # הנחיית מערכת
    messages=[
        {"role": "user", "content": "...שאלה כאן"}
    ]
)
print(message.content[0].text)
```

עלות API:

Claude מחייב לפי **טוקנים** (כ-750 מילים = ~1000 טוקנים). לחקר אקדמי: \$5-15 לחודש בשימוש מתון.

```
In [ ]: # =====
# לניתוח טקסטים היסטוריים Claude API-שימוש ב
# =====

def analyze_with_claude(text, task, api_key=None):
    """
    Claude API. ניתוח טקסט עם

    פרמטרים:
        text      : הטקסט לניתוח
        task      : המשימה ('summarize', 'classify', 'ner', 'translate')
        api_key   : מפתח Anthropic API

    string-כ Claude מחזיר: תגובת
```

```

מחזיר תגובה מדומה להדגמה, api_key הערה: ללא
"""

# הנחיות מערכת לפי משימה
SYSTEM_PROMPTS = {
    'summarize': """אתה עוזר מחקרי המתמחה במדעי הרוח""",
    'classify': """אתה מומחה לסיווג טקסטים בתחום מדעי הרוח""",
    'ner': """אתה מומחה להיסטוריה ישראלית ויהודית""",
    'translate': """אתה מתרגם המתמחה בטקסטים עתיקים""",
}

# הנחיות משתמש לפי משימה
USER_PROMPTS = {
    'summarize': f"סכם ב-3 משפטים:\n\n{{text}}",
    'classify': f"סווג לקטגוריות (JSON):\n\n{{text}}",
    'ner': f"חלץ ישויות בשם (JSON):\n\n{{text}}",
    'translate': f"תרגם לעברית מודרנית\n\n{{text}}"
}

if not api_key:
    # מצב הדגמה - תגובות מדומות
    return simulate_llm_response(USER_PROMPTS.get(task, text))

try:
    import anthropic
    client = anthropic.Anthropic(api_key=api_key)

    message = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=512,
        system=SYSTEM_PROMPTS.get(task, ""),
        messages=[{"role": "user", "content": USER_PROMPTS.get(task, text)}]
    )
    return message.content[0].text

except Exception as e:
    return f"API שגיאת: {e}"

# -- הדגמה --
sample_texts = [
    "חפירות בתל מגידו חשפו 26 שכבות התיישבות מ-7000 שנה של היסטוריה" +
    "הממצאים כוללים ארמונות מהתקופה הכנענית, אורוות מהתקופה האיסורית" +
    "ותעלת מים מתוחכמת. האתר זוהה עם המגידו המקראית",

    "דוד המלך כבש את ירושלים מהיבוסים וייסד אותה לבירת ממלכת ישראל" +
    "בנו שלמה בנה את בית המקדש הראשון, שהיה מרכז פולחן עד חורבנו" +
    "בידי נבוכדנאצר מלך בבל בשנת 586 לפנה"ס",
]

```

```

print("👑 AI (מצב הדגמה):")
print("=" * 60)

for i, text in enumerate(sample_texts, 1):
    print(f"\n📄 טקסט {i}: {text[:60]}...")
    print()

    for task in ['summarize', 'classify']:
        print(f"📋 משימה: {task}")
        response = analyze_with_claude(text, task)
        print(f"📄 תגובה: {response.strip()[:150]}...")
        print()

    print("-" * 50)

print("\n💡 API לשימוש עם")
print("    analyze_with_claude(text, 'summarize', api_key='sk-...')"
```

חלק ד: סיווג טקסטים עם AI

גישות לסיווג:

גישה	כלים	יתרון	חסרון
Zero-shot	LLM (Claude/GPT)	ללא נתוני אימון	פחות מדויק
Few-shot	LLM + דוגמאות	מאוזן	דורש דוגמאות
Fine-tuning	BERT/DistilBERT	הכי מדויק	דורש נתונים + GPU
כלאיים	Rule-based + ML	מהיר, ניתן לפרש	מורכב לבנות

```

In [ ]: # =====
# Pipeline סיווג טקסטים (Zero-shot + Few-shot)
# =====

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import numpy as np

# -- נתוני אימון לדוגמה --
TRAINING_DATA = [
    # ארכיאולוגיה
    ("חפירה בתל מגידו חשפה ממצאים מהתקופה הכנענית", "ארכיאולוגיה"),
    ("ממצאים ארכיאולוגיים כוללים חרסים ומטבעות עתיקות", "ארכיאולוגיה"),
    ("שכבות ההתיישבות מציגות תרבויות שונות לאורך אלפי שנים", "ארכיאולוגיה"),
    ("אתר ארכיאולוגי ים המלח חשף מגילות וכתבים עתיקים", "ארכיאולוגיה"),
    ("רת מצילה ברחוב ירושלים גילתה מקוואות מהתקופה ההרודיאנית", "ארכיאולוגיה"),
    ("ממצאי ברזל מהמאה עשר לפנה ספירה מוכיחים פעילות מתכתית", "ארכיאולוגיה")
]
```



```

# היסטוריה
("דוד המלך ייסד את ירושלים לבירת ממלכת ישראל",
"חורבן הבית הראשון בידי הבבלים שינה את ההיסטוריה היהודית",
"ממלכת חשמונאים שלטה ביהודה מהמאה השנייה לפנה ספירה",
"גירוש יהודי ספרד בשנת ד אלפים ורנב השפיע על יהדות כולה",
"עליות לארץ ישראל בראשית המאה העשרים שינו את הדמוגרפיה",
"קום המדינה בתש ח היה נקודת מפנה בהיסטוריה היהודית",
"היסטוריה"),

# מדעי הרוח הדיגיטליים
("DH", "ניתוח קורפוסים גדולים דורש כלים חישוביים מתקדמים"),
("DH", "מפשטת מחקר GIS-ויזואליזציה של נתונים ארכיאולוגיים ב"),
("DH", "מידול נושאים חושף דפוסים נסתרים בטקסטים היסטוריים"),
("DH", "מאפשר דיגיטציה של כתבי יד וטקסטים ישנים OCR"),
("DH", "רשתות חברתיות של דמויות היסטוריות ניתנות לניתוח חישובי"),
("DH", "בינה מלאכותית מסייעת לפרשנות כתבות עתיקות"),

]

# פיצול לאימון ובדיקה
texts, labels = zip(*TRAINING_DATA)
X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.3, random_state=42
)

# -- מודל TF-IDF + Naive Bayes --
pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(analyzer=lambda x: list(re.sub(r'^\u05D0-\u05
    ('clf', MultinomialNB(alpha=0.1))
])

pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)

print("📊 תוצאות הסיווג")
print("=" * 55)
print(classification_report(y_test, y_pred, zero_division=0))

# -- בדיקת טקסטים חדשים --
print("\n🔍 בדיקת טקסטים חדשים:")
new_texts = [
    "ממצאים ארכיאולוגיים חדשים בגיא בן הינום",
    "Python ו NLTK- ניתוח טקסטים עם",
    "מרד המכבים נגד הסלאוקידים"
]

for text in new_texts:
    pred = pipeline.predict([text])[0]
    proba = pipeline.predict_proba([text])[0]
    classes = pipeline.classes_

    print(f"\n '{text[:45]}...'")
    print(f" -> {pred} ", end="")
    for cls, prob in zip(classes, proba):
        print(f"| {cls}: {prob:.2f}", end="")
    print()

```

חלק ה: שיקולים אתיים – AI ומדעי הרוח

שאלות אתיות מרכזיות:

1. הטיית (Bias)

- AI מאומן על טקסטים אנגליים -> הטיה מערבית
- ייצוג-חסר של תרבויות לא-מערביות
- מגדר, גזע, מעמד – כולם מיוצגים בהטיות האימון

2. Hallucinations

- AI ממציא עובדות שנשמעות אמיתיות
- **מסוכן** במיוחד בהיסטוריה ובארכיאולוגיה
- **חובה** לאמת כל עובדה מול מקורות ראשוניים

3. שקיפות וסיוע

- כיצד לציין שימוש ב-AI במחקר אקדמי?
- מהי הגבול בין "עזרה" ל"מחקר שנעשה על ידי AI"?

4. פרטיות

- שליחת מסמכים ל-API = שיתוף עם חברה חיצונית
- מסמכים ארכיוניים רגישים?

המלצות לשימוש אחראי:

- בדקו עובדות של AI מספק
- צינו שימוש ב-AI בפרסומים אקדמיים
- השתמשו ב-AI כ"עוזר" ולא כ"מחבר"
- שמרו על שיפוט ביקורתית

```
In [ ]: # =====
# Hallucinations-בדיקת עובדות ו
# =====

# (Fact Checking) מסד ידע בסיסי לאימות
KNOWN_FACTS = {
    'מגידו': {
        'מיקום': 'עמק יזרעאל',
        'תקופות': ['כנענית', 'ברונזה', 'ברזל'],
        'חוקרים': ['גוטליב שומכר', 'קלרנס פישר']
    },
    'ירושלים': {
        'ייסוד': 'לפחות 3500 שנה',
        'בניין_מקדש': 'שלמה המלך',
        'חורבן_ראשון': '586 לפנה"ס'
    }
}
```

```

},
'גניזה קהירית': {
    'מיקום': 'קהיר, מצרים',
    'תקופה': 'המאה ה-9 עד ה-20',
    'גודל': 'כ-300,000 קטעים'
}
}

def verify_claim(claim, fact_db=None):
    """
    בדיקת עובדה מול מסד ידע.

    Wikidata API- בפרויקט אמיתי: ניתן לחבר ל
    """
    if fact_db is None:
        fact_db = KNOWN_FACTS

    warnings = []

    # חיפוש ישויות ידועות
    for entity, facts in fact_db.items():
        if entity in claim:
            # בדיקת עקביות
            for key, value in facts.items():
                if isinstance(value, str) and value in claim:
                    pass # עובדה תואמת

    return warnings

# -- Hallucinations הדגמת זיהוי --
print("🔍 Hallucinations: הדגמת בעיות")
print("=" * 60)

ai_outputs = [
    {
        'prompt': 'מה ידוע על תל מגידו?',
        'response': '1903-הוא אתר ארכיאולוגי בעמק יזרעאל, שנחפר לראשונה ב-1903',
        'correct': True,
        'note': 'י נכון'
    },
    {
        'prompt': 'מתי נחרב בית המקדש הראשון?',
        'response': '586 לפנה"ס בידי בוכדנאצר מלך בבל',
        'correct': True,
        'note': 'י נכון'
    },
    {
        'prompt': 'מי ביצע את חפירות הגניזה הקהירית?',
        'response': '1890 על ידי סולומון שכטר ופרנקו מורטי',
        'correct': False,
        'note': 'ו מורטי הוא חוקר ספרות, לא ארכיאולוג. הדמות הנוספת היא אחרת'
    },
    {
        'prompt': 'מה גודל הגניזה הקהירית?',
    }
]

```

```

        'response': 'הגניזה הקהירית מכילה כ-10,000 קטעי מסמכים.',
        'correct': False,
        'note': 'שגוי! ✗ הגניזה מכילה כ-300,000 קטעים - פי 30 מהתשובה'
    }
]

for item in ai_outputs:
    status = "✅" if item['correct'] else "✗"
    print(f"\n{status} שאלה: {item['prompt']}")
    print(f"    תגובת AI: {item['response'][:80]}...")
    print(f"    ⚠️ {item['note']}")

print("\n🔑 מסקנה:")
print("    מספק מול מקורות אמינים AI תמיד בדקו עובדות ש")

```

```

In [ ]: # =====
# מחקר מדעי הרוח + AI :תהליך עבודה מעשי
# =====

def humanities_research_pipeline(texts, research_question, use_api=False, api_key=None):
    """
    עם מתודולוגיה הומניסטית AI מחקרי שמשלב Pipeline.

    שלבים:
    1. קריאה מרחוק (כמות)
    2. סיכום וסיווג AI
    3. זיהוי טקסטים מעניינים לקריאה קרובה
    4. קריאה קרובה אנושית

    פרמטרים:
        texts          : רשימת טקסטים
        research_question : שאלת המחקר
        use_api        : אמיתי API-האם להשתמש ב
        api_key        : מפתח API

    עם תוצאות DataFrame מחזיר
    """
    results = []

    print(f"📖 שאלת מחקר: {research_question}")
    print(f"    (= * 60)")

    for i, text in enumerate(texts, 1):
        print(f"\n📖 מסמך {i}/{len(texts)}: {text[:50]}...")

        # שלב 1: מטה-נתונים בסיסיים
        word_count = len(text.split())

        # שלב 2: AI - סיכום
        summary = analyze_with_claude(text, 'summarize', api_key)

        # שלב 3: AI - סיווג
        classification = analyze_with_claude(text, 'classify', api_key)

        # שלב 4: ציון "עניין" פשוט
        interesting_keywords = ['ראשון', 'מרכזי', 'נדיר', 'חשף', 'גילה', 'די', 'רע', 'אנוש', 'מפתח', 'מרכזי', 'נדיר', 'חשף', 'גילה', 'די', 'רע', 'אנוש', 'מפתח']
    
```

```

interest_score = sum(1 for kw in interesting_keywords if kw in text)

results.append({
    'מסמך': i,
    'טקסט_קצר': text[:80] + '...',
    'מילים': word_count,
    'סיכום_AI': summary.strip()[:200],
    'סיווג_AI': classification.strip()[:100],
    'ציון_עניין': interest_score
})

print(f" ✓ עניין | עובד: {interest_score}/10")

df = pd.DataFrame(results)
return df

# -- הרצה --
sample_for_pipeline = [
    "בתל מגידו חשפו 26 שכבות התיישבות. הממצאים חשובים ומרכזיים להבנת התקופה",
    "דוד המלך גילה ממלכה גדולה שייסד ירושלים לבירת ממלכת ישראל",
    "כלים דיגיטליים ייחודיים לניתוח הגניזה הקהירית"
]

df_results = humanities_research_pipeline(
    sample_for_pipeline,
    research_question="הם הנושאים המרכזיים בכתיבה ארכיאולוגית-היסטורית עברית"
)

print("\n📋 Pipeline:")
display(df_results[['מסמך', 'מילים', 'ציון_עניין', 'טקסט_קצר']])

df_results.to_csv('ai_analysis_results.csv', index=False, encoding='utf-8-si
print("\n📄 ai_analysis_results.csv")

```

```

In [ ]: # =====
# AI ויזואליזציה של תוצאות
# =====

# נתוני דוגמה לניתוח
demo_results = {
    'היסטוריה': 'DH', 'קטגוריה': 'ארכיאולוגיה', 'היסטוריה': 'ארכיאולוגיה',
    'היסטוריה': 'DH', 'קטגוריה': 'ארכיאולוגיה', 'היסטוריה': 'ארכיאולוגיה',
    '0.89, 0.93, 0.86, 0.78, 0.91, 0.82, 0.95, 0.75, 0.88, 0.92]': '0.89, 0.93, 0.86, 0.78, 0.91, 0.82, 0.95, 0.75, 0.88, 0.92]':
    '140, 160, 110, 130, 180, 95, 150, 200, 85, 120]': '140, 160, 110, 130, 180, 95, 150, 200, 85, 120]':
}

df_demo = pd.DataFrame(demo_results)

# גרפים
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
colors = {'ארכיאולוגיה': '#e74c3c', 'היסטוריה': '#2980b9', 'DH': '#27ae60'}

# 1. התפלגות קטגוריות
cat_counts = df_demo['קטגוריה'].value_counts()
cat_colors = [colors[c] for c in cat_counts.index]
axes[0].pie(cat_counts, labels=cat_counts.index, colors=cat_colors,

```

```

        autopct='%1.0f%%', startangle=90)
axes[0].set_title('התפלגות קטגוריות', fontsize=12, fontweight='bold')

# 2. ציון ביטחון לפי קטגוריה
for cat, color in colors.items():
    cat_data = df_demo[df_demo['קטגוריה'] == cat]
    axes[1].scatter(range(len(cat_data)), cat_data['ציון_ביטחון'],
                    c=color, label=cat, s=100, alpha=0.8)
axes[1].set_ylim(0, 1)
axes[1].set_ylabel('ציון ביטחון', fontsize=11)
axes[1].set_title('ציון ביטחון הסיווג', fontsize=12, fontweight='bold')
axes[1].legend(fontsize=9)
axes[1].axhline(y=0.7, color='orange', linestyle='--', alpha=0.7, label='0.7')
axes[1].grid(True, alpha=0.3)

# 3. ביטחון מול אורך
cat_color_list = [colors[c] for c in df_demo['קטגוריה']]
scatter = axes[2].scatter(df_demo['אורך_טקסט'], df_demo['ציון_ביטחון'],
                        c=cat_color_list, s=120, alpha=0.8, edgecolors='w')
axes[2].set_xlabel('אורך טקסט (מילים)', fontsize=11)
axes[2].set_ylabel('ציון ביטחון', fontsize=11)
axes[2].set_title('ביטחון לעומת אורך', fontsize=12, fontweight='bold')
axes[2].grid(True, alpha=0.3)

# מקרא
from matplotlib.patches import Patch
legend_elements = [Patch(facecolor=c, label=cat) for cat, c in colors.items()]
axes[2].legend(handles=legend_elements, fontsize=9)

plt.suptitle('סיווג טקסטים - AI ניתוח תוצאות', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('01_ai_results.png', dpi=150, bbox_inches='tight')
plt.show()
print(f"📁 01 הגרף נשמר: ai_results.png")

```

```

In [ ]: # =====
# שמירת הנתונים
# =====

print(f"📁 שומר נתונים...")

if 'df_results' in dir():
    df_results.to_csv('ai_analysis_results.csv', index=False, encoding='utf-8')
    print(f"✅ ai_analysis_results.csv")

print(f"\n🎉 מחברת 5 הושלמה!")
print()
print(f"📄 אמיתי AI-משאבים לשימוש ב")
print(f"➔ Claude API: https://console.anthropic.com/")
print(f"➔ AlephBERT: https://huggingface.co/onlplab/alephbert-base")
print(f"➔ Gemini API: https://ai.google.dev/")
print()
print(f"⚠️ תמיד:")
print(f"🔍 1. מספק AI בדקו עובדות ש")

```

```
print("בפרסומים AI-ציינו שימוש ב. 2")
print("שמרו על שיפוט ביקורתי. 3")
```

תרגילים

תרגיל 1 – בסיסי

כתבו 3 prompts שונים לאותה משימה (סיכום טקסט היסטורי).
השוו את התוצאות – מה ה-prompt שנתן את התוצאה הטובה ביותר?

תרגיל 2 – בינוני

הוסיפו 10 טקסטים חדשים לנתוני האימון בפונקציית הסיווג.
שפרו את הדיוק ומדדו שינוי בתוצאות ה-classification_report.

תרגיל 3 – מתקדם

השיגו מפתח API חינמי (Claude/Gemini) והריצו ניתוח אמיתי
על 20 ערכים מהקורפוס שנאסף במחברת 1.
השוו: AI לעומת TF-IDF בסיווג – מי מדויק יותר?

משאבים

- [Anthropic Claude API](#)
- [OpenAI API](#)
- [AlephBERT](#)
- [Bender et al. \(2021\). On the Dangers of Stochastic Parrots. FAccT](#)
- [AI Ethics in Digital Humanities](#)

משימת הבית 🏠

מה להגיש:

1. **Prompt מקורי** – כתבו פרומפט לניתוח טקסט היסטורי / ארכיאולוגי משלכם
2. **תגובת Gemini** – הריצו את הפרומפט שלכם עם `call_gemini()` והדביקו את התגובה
3. **הערכה** – כתבו 2-3 משפטים: מה טוב בתגובה? מה הייתם משפרים בפרומפט?

כיצד להגיש:

1. **File → Save a copy in Drive** – שמרו עותק אישי
2. **File → Download → Download.ipynb** – הורידו את הקובץ

עצה: אם נתקעתם – שאלו את Gemini! 🤖

פתחו טאב חדש ב-[Google AI Studio](#) ושאלו:

DH. אני יוצרת פרומפט לניתוח טקסט היסטורי בשיעור
"הפרומפט שלי הוא: "[הדביקי כאן]
?מה אפשר לשפר בו